

RAP Cloud 課程 1：環境介紹 + CDS View Entity

Lecture

這門課在教什麼、跟舊 RAP 課程的關係

這門課是 `src/ABAP_Training_RAP/` (rap01–rap09 , 已正式結案) 的延伸篇，環境換成一個真正啟用「ABAP Cloud 語言版本」的 SAP BTP ABAP Environment (Trial)。不重講舊課程已經教過的東西 (Managed/Unmanaged 概念、RAP 五層架構、BDEF 基本語法)——這一課的重點只有一件事：搞懂這個新環境的操作方式，跟舊環境比起來哪裡不一樣，然後動手建出這門課第一組物件。

環境總覽：這個 Cloud 環境的操作方式，完全是另一套

	舊 RAP 課程 (On-Premise)	這門課 (Cloud)
連線方式	Eclipse Plugin + 本機 <code>adt-rfc-bridge</code> (RFC 轉 HTTP)	VS Code 擴充套件 <code>vscode-abap-remote-fs</code> 內建的 MCP Server
MCP Server 名稱	<code>sap-adt</code>	<code>abap-remote-fs</code> , <code>connectionId</code> 固定是 <code>abap_cloud</code>
建立新物件	Claude 可以直接用 <code>sap_create_object</code> / curl 打 ADT API 建立	Claude 端的建立功能目前故障 (不分套件、不分物件型別，一律卡在同一類反序列化錯誤)，一律要你在 Eclipse ADT 手動建立空殼
讀取 / 修改 / 啟用既有物件	Claude 可以	Claude 可以 (<code>get_abap_object_lines</code> / <code>replace_string_in_abap_object</code> / <code>abap_activate</code> / <code>get_abap_diagnostics</code>)，這部分完全正常
無頭執行驗證	<code>programrun</code> (跑報表看 <code>WRITE</code> 輸出)	沒有對應工具，改用 ABAP Unit (<code>run_unit_tests</code>) 驗證 RAP CUD / Determination / Validation / Action，用 <code>execute_data_query</code> 驗證資料
系統性質	使用者專屬的正式開發系統	公開 / 多人共用的社群 Trial 系統——套件清單裡有大量明顯屬於其他學員的物件 (不同語言、不同線上課程的練習內容)

本課程固定的分工模式：每一課只要牽涉到「建立新物件」，一律先請你在 Eclipse ADT 建好空殼 (只要填 Name / Description / Package，不用填內容)，跟我說一聲之後，我接手寫入完整內容、語法檢查、啟用、驗證。這比舊課程的分工範圍更廣 (舊課程只有 Service Binding 等少數物件需要手動建立)，但空殼建好之後，Claude 這邊能做的事其實更完整、更省事 (不用清鎖、不用手動組 XML 打 curl，直接 find-and-replace + 一個指令啟用)。

CDS View Entity 語法：新式 vs. 舊式逐項對照

舊課程受限於系統版本，一路只能用舊式 `define view` (`.claude/rules/sap-adt-mcp.md` 第 40.2 節)。這個 Cloud 環境版本完全足夠，可以用官方目前主推的新式語法：

```

" 舊式 ( On-Premise 1909 · 舊課程全程使用 )
@AbapCatalog.sqlViewName: 'ZIRC01TASK'
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@ObjectModel.compositionRoot: true
define root view ZI_RC01_TASK
  as select from zrc01_task
{
  key task_id,
  ...
}

" 新式 ( 這個 Cloud 環境 · 這門課全程使用 )
@AccessControl.authorizationCheck: #NOT_REQUIRED
define root view entity ZI_RC01_TASK
  as select from zrc01_task
{
  key task_id,
  ...
}

```

三個具體差異：

	舊式 define view	新式 define view entity
關鍵字	沒有 entity	define root view entity (多了 entity)
@AbapCatalog.sqlViewName	必填 ，底層要另外產生一個 ≤16 碼的實體 SQL View 物件	不需要 ——新式語法底層機制不同，沒有這個「純技術輔助物件」的概念
@AbapCatalog.preserveKey / @ObjectModel.compositionRoot	都要明寫，缺 preserveKey 會直接啟用失敗 (舊課程 rap01 踩過的坑)	都不需要 ——「這是 RAP BO 根節點」這件事，直接由 define root view entity 的 root 關鍵字表達，不用另外兩個 annotation 重複宣告

這是查證既有物件得到的結論，不是憑官方文件推測：本課程開課前查證階段 (見 memory **cloud-rap-exploration**) 讀過這系統既有標準/學員物件 **Z0067_AGENCY**：

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: ' Agency Root View'
@Metadata.allowExtensions: true
define root view entity Z0067_AGENCY
  as select from /dmo/agency
{ ... }

```

ZI_RC01_TASK (這一課的物件) 就是照這個已驗證過的實際範例設計，不是照抄官方文件字面 (官方文件的範例有時會跟特定系統版本的實際支援程度有落差，這點舊課程已經踩過好幾次坑)。

這一課的物件設計

情境延續舊課程 **rap02** 的「**Task 管理**」主題，在這個環境重新建一版：

```
" Table
@EndUserText.label : 'RC01 Task Root Table'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table zrc01_task {
  key client      : abap.clnt not null;
  key task_id     : abap.char(10) not null;
  description     : abap.char(100);
  status         : abap.char(1);
  priority        : abap.char(1);
  due_date       : abap.dats;
  created_at      : abap.utclong;
  created_by     : abap.char(12);
}
```

```
" CDS View Entity
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RC01 Task Interface View'
@Metadata.allowExtensions: true
define root view entity ZI_RC01_TASK
  as select from zrc01_task
{
  key task_id,
  description,
  status,
  priority,
  due_date,
  created_at,
  created_by
}
```

三個跟舊課程不一樣、值得注意的地方：

1. ⚠️ 這一課刻意不幫每個欄位建 **Domain + Data Element**，直接用內建型別 (`abap.char` / `abap.dats` / `abap.utclong`) —— 這是有意的簡化，不是忘記硬性規則 (`.claude/rules/sap-adt-mcp.md` 記載的「**DDIC 欄位一律要引用 Data Element**」)。原因：這個環境「建立新物件」這一步全部要走 Eclipse，如果比照舊課程規格幫 5 個業務欄位都建一組 Domain + DE，一課就要多跑 10 次 Eclipse 建立

空殼的來回，這門課的重點是 RAP/CDS Entity 機制本身，DDIC 欄位規範舊課程已經教得很紮實，這裡不重複。

2. **Client** 欄位型別是 **abap.clnt**，不是舊課程用的 **mandt** (**Data Element**) ——這是 Eclipse 「New Database Table」精靈在這個 Cloud 環境自動產生的骨架就長這樣，不是我們自己選的：Cloud 系統的標準骨架偏好用內建型別 **abap.clnt** 表示 Client 欄位，不像 On-Premise 系統習慣引用 **MANDT** 這個標準 Data Element。兩者效果一樣（都是 Client 欄位型別），只是型別來源不同。
3. 時間戳記欄位用 **abap.utclong**，不是舊課程用的 **TIMESTAMPL** (**Data Element**) ——**UTCLONG** 是 ABAP 7.54 之後、Cloud 世代慣用的時間型別（8 byte 二進位，精度到奈秒），舊課程的 **TIMESTAMPL** (DEC21.7) 是比較舊的表示法。這一課先只是建立欄位，還沒有自動填值的邏輯（那是舊課程 rap05 的 Determination 主題，等這門課後面幾課會重新示範一次「Cloud 版」的寫法）。

⚠ 啟用 **Table** 時出現一個新的 **Warning**，這系統版本才會看到：

WARNING Line 6, Col 14: Key must have the type Inverted Individual on the database

這是 HANA Cloud 資料庫層對 Column Store 索引型別的建議（HANA 的 Primary Key 可以指定不同的索引結構：Inverted Value / Inverted Hash / Inverted Individual），只是警告、不擋啟用，這一課先如實記錄這個現象，不深入查證（舊課程 On-Premise 系統完全沒出現過這個警告，這是 Cloud/新版 HANA 才有的提示，值得知道有這回事，但不影響這一課的驗收）。

Eclipse ADT Step by Step：建立空殼（本課實際操作記錄）

1. 對著套件 **ZRAPCLOUD** 右鍵 → New Database Table，Name **ZRC01_TASK**，Description、Package 照填，Transport Request 選現有或新建皆可。
2. 對著 **ZRAPCLOUD**（或剛建的 Table）右鍵 → New → Other ABAP Repository Object → **Data Definition**，Name **ZI_RC01_TASK**。
3. **Templates** 畫面選「**Define Root View Entity**」——這跟舊課程完全相反：舊課程要求你「避開任何帶 Entity 字樣的模板」，這門課反而就是要選它，因為這個環境的編譯器支援新式語法。
4. 兩個都只建空殼（Eclipse 自動產生的預設骨架，不用手動改），Finish 之後跟 Claude 說一聲，由 Claude 接手寫入完整內容（**replace_string_in_abap_object**）、語法檢查（**get_abap_diagnostics**）、啟用（**abap_activate**）。

學習目標

- 能講出這個 Cloud 環境跟舊 On-Premise 環境在連線方式、建立物件、無頭驗證工具三方面的具體差異
- 知道本課程固定的分工模式：建立新物件一律要在 Eclipse ADT 手動做，Claude 接手後續所有動作
- 能寫出新式 **define root view entity** 語法，知道它比舊式 **define view** 少了 **@AbapCatalog.sqlViewName** / **@AbapCatalog.preserveKey** / **@ObjectModel.compositionRoot** 三個東西，以及各自為什麼不需要
- 知道 Cloud 環境的 Table 骨架偏好用 **abap.clnt** / **abap.utclong** 這類內建型別，跟 On-Premise 慣用的 **MANDT** / **TIMESTAMPL** Data Element 是同樣效果、不同來源
- 知道這一課刻意不建 Domain / Data Element 的原因（環境限制造成的協調成本，不是忽略硬性規則）
- 能用 **execute_data_query** 對 CDS View Entity 下一句基本 **SELECT** 驗證查詢正常

物件清單

物件	名稱	型別
DDIC Table	ZRC01_TASK	TABL/DT
CDS Root View Entity	ZI_RC01_TASK	DDLS/DF

套件：ZRAPCLOUD (使用者建立) 。兩個物件都已在 Eclipse 建立空殼、Claude 寫入完整內容並啟用成功 (`get_abap_diagnostics` 確認無錯誤 / 無警告，`execute_data_query` 確認可正常查詢，目前資料是空的，資料填入是下一課 Managed BDEF 的內容) 。

動手練習

輪到你了：用同樣的流程，在 Eclipse 建一組全新的小型 Table + CDS View Entity (情境自訂，例如「任務分類」ZRC01_CATEGORY / ZI_RC01_CATEGORY，只要 2-3 個簡單欄位，不需要跟 ZRC01_TASK 有任何關聯——關聯的部分留到 rc06 的 Composition 主題) ：

1. Eclipse 建立空殼 (Table + CDS View Entity 各一個)，套件 ZRAPCLOUD
2. 建好後跟我說，我會幫你確認 Templates 有沒有選對 (Define Root View Entity)、欄位型別要用內建型別還是有沒有更合適的既有標準物件可以參考
3. 我可以幫你將內容寫進去、啟用、驗證 (或你想自己動手寫內容也可以，寫完貼給我核對)

驗證方式

Table / CDS View 不是可執行的程式，這一課的驗證重點是語法正確 + 成功啟用 + 查得到資料：

1. `get_abap_diagnostics` 確認兩個物件都無錯誤 (Table 有 1 個不影響啟用的 Warning，已記錄原因)
2. `abap_activate` 兩個物件都回報 `Activation successful`
3. `execute_data_query` 對 ZI_RC01_TASK 下 `SELECT task_id, description, status FROM ZI_RC01_TASK`，正常回傳 0 筆 (資料表目前是空的，欄位結構正確)

⚠ **重要澄清：**`execute_data_query` 是 MCP 工具的名稱，`SELECT ... FROM ZI_RC01_TASK` 不是 EML、也不是真的 ABAP 程式碼——這一點容易搞混，先講清楚：

- `execute_data_query` 是 `abap-remote-fs` 這個 MCP Server 提供的工具 (就像 `get_abap_object_lines` / `abap_activate` 一樣，是 Claude 這邊呼叫的一個「動作」)，作用是「臨時對這個系統下一句查詢、把結果顯示出來」——角色等同舊課程 On-Premise 系統的 ADT Data Preview / `datapreview/freestyle` API，或者你熟悉的 SE16 資料瀏覽器，純粹是查資料用的輔助工具，本身不是 ABAP 語言的一部分。
- 傳給這個工具的 `SELECT task_id, description, status FROM ZI_RC01_TASK` 字串，是這個工具自己接受的一種類 SQL 查詢語法 (細節見 `get_abap_sql_syntax` 工具說明，跟標準 Open SQL 有一些差異，例如 `ORDER BY field DESCENDING` 不能寫 `DESC`)，這句話從頭到尾不會被編譯成一支 ABAP 程式，也不會出現在任何 `.abap` 原始碼檔案裡，純粹是「請這個系統幫我查一下、把結果印出來看」的臨時指令。
- **EML (Entity Manipulation Language)** 是完全不同的東西——EML 是真正的 ABAP 語言關鍵字 (`READ ENTITIES OF ... / MODIFY ENTITIES OF ... / COMMIT ENTITIES`)，只能寫在真正的 ABAP 程式 / 類別原始碼裡，要先啟用、再執行 (或像 rc02 那樣包在 ABAP Unit 測試方法裡跑)，是這門課後面驗證 RAP CUD / Determination / Validation / Action 行為的正式管道。`execute_data_query` 沒辦法呼叫 EML，也沒辦法測試 BDEF 的行為邏輯，它只能做「查資料庫/CDS View 現在長怎樣」這種靜態查詢。

- 這一課只用得到 `execute_data_query`，因為 rc01 還沒有 BDEF、沒有任何「行為」可以測試，只需要確認 CDS View 查詢得到正確的欄位結構；**rc02** 開始會大量用到 **EML**（見下一課），兩者的分工是：
`execute_data_query` = 查資料現況，EML = 驅動 RAP BO 的行為（新增/修改/刪除/呼叫 Action）。

思考題

1. 新式 `define view entity` 不需要 `@AbapCatalog.sqlViewName`，這代表底層完全沒有對應的資料庫實體物件了嗎？（提示：想想官方文件說 SQL View Name 是「purely technical helper construct」，這句話本身暗示了什麼）
2. 這一課的 Table 用 `abap.clnt`、CDS View 完全沒有把 Client 欄位列進欄位清單——這個行為（Client 欄位自動處理、不用曝露在輸出）跟舊課程 rap02 教過的 CDS View Client Handling 機制是同一套邏輯嗎，還是新式語法有不同的處理方式？
3. 如果之後（rc02）要幫這個 Table 加 Managed BDEF，`persistent table zrc01_task` 這個子句在新式語法的 BDEF 裡寫法會不會不一樣？（先想一想，不用現在查證，rc02 會正式教）

答案

見 `zrc01_task.tabl.abap`、`zi_rc01_task.ddls.abap`。SAP 端物件：**ZRC01_TASK**（Table）、**ZI_RC01_TASK**（CDS View Entity），套件 **ZRAPCLOUD**。動手練習（任務分類 Table + CDS View）由你在 Eclipse 動手建立，沒有固定答案快照——建好後跟我核對即可。